ITM SKILLS UNIVERSITY

School of Future Tech

Programming Assignment Report

on

Programming Assignment 1: Two-Pointer Technique

Removing Duplicates from a Sorted Array and Pair Sum in a Sorted Array

by

Aaryan Keni
150096725197

B.Tech CSE 2025-29 | Data Structures and Algorithms II (DSA-II)
Semester III | AY 2026-27

Index

1. Introduction to the Assignment.

2. Problem Statement / Assignment Background (Abstract).

3. Assignment Design.

4. Methods, Frameworks & Tools Applied in the Assignment.

5. Implementation Details and Snapshots.

6. Assignment Results and Conclusion.

7. References

1. Introduction to the Assignment

Many array problems look as if they need two nested loops, yet become much simpler when the array is **sorted**. The **two-pointer technique** keeps two indices into the array and moves them according to a simple rule, so that every element is looked at only a constant number of times. This assignment applies the technique to two classic problems on sorted integer arrays: **removing duplicates in place** with a slow and a fast pointer, and **finding a pair with a given sum** with pointers at opposite ends.

For each problem, a straightforward reference (or brute-force) solution and an optimized two-pointer solution are written in **C++, Java and Python**. The two versions are compared on time and auxiliary-space complexity, traced with dry-run tables and checked with normal, boundary, duplicate-heavy and no-solution test cases.

The assignment shows that knowing how the data is arranged can change the cost of a solution: the same answers are produced, but the optimized versions need one pass instead of a quadratic number of comparisons, or no extra array at all.

Learning Objectives

- Apply pointer-based reasoning to exploit the sorted order of an array.
- Use slow/fast pointers for in-place duplicate removal and opposite-end pointers for pair sum.
- Explain why and when each pointer moves, and why the optimized algorithm is correct.
- Compare brute-force and optimized solutions using time and auxiliary-space complexity.
- Validate solutions using normal, boundary, duplicate-heavy and no-solution test cases.

2. Problem Statement / Assignment Background (Abstract)

Background

Sorted arrays have a useful property: equal values sit next to each other, and moving an index to the right can only make a value larger or equal. Both problems in this assignment rely on exactly these two facts. A solution that ignores them has to compare every element with every other element or keep a second copy of the data.

Problem Statement

Part A: Remove duplicates from a sorted array

Given a non-decreasing sorted integer array, modify the array **in place** so that each distinct value appears once in the first k positions. Return k, the number of distinct values. The order of the distinct values must remain unchanged. Values after index k-1 may be ignored; the array does not need to physically shrink. Empty and one-element arrays must be handled.

Example: input `[1, 1, 2, 2, 2, 3, 4, 4]` gives the unique prefix `[1, 2, 3, 4]` and k = 4.

Part B: Pair sum in a sorted array

Given a sorted integer array and a target T, return the two indices i and j (i < j) such that a[i] + a[j] = T. If no pair exists, return (-1, -1). The same element must not be used twice, and if several pairs exist, any valid pair is acceptable.

Example: input `[1, 2, 4, 7, 11, 15]` with target 15 gives indices (2, 4), values (4, 11).

Objectives

- Implement a reference solution and an in-place slow/fast pointer solution for duplicate removal.
- Implement a brute-force nested-loop solution and an opposite-end two-pointer solution for pair sum.

- Explain pointer initialization, loop condition, comparison condition, pointer update and termination for each optimized algorithm.
- Provide dry-run tables, a test-case table, a complexity comparison table and screenshots of the code and its output.
- Submit the work in C++, Java and Python.

Abstract

This report presents the design, implementation and testing of two two-pointer algorithms on sorted integer arrays. For **duplicate removal**, a reference solution that builds a separate list (O(N) time, O(N) extra space) is compared with a **slow and fast pointer** solution that works in place (O(N) time, O(1) extra space). For **pair sum**, a **nested-loop** solution (O(N^2) time) is compared with an **opposite-end pointer** solution (O(N) time, O(1) extra space). All four algorithms were written in C++17, Java and Python and give identical results. The sample input `[1, 1, 2, 2, 2, 3, 4, 4]` gives k = 4 with prefix `[1, 2, 3, 4]`, and `[1, 2, 4, 7, 11, 15]` with target 15 gives the pair (2, 4).

Note on figures: the code screenshots in Section 5 are taken from the source files in Visual Studio Code. The outputs, dry-run tables and test results in this report come from running those same functions. One expected value in the assignment's test table does not match its own input; this is explained in Section 5.2.

3. Assignment Design

Each problem is designed in the same four steps:

1. **Interpretation:** restate the input, output and constraints.
2. **Reference version:** write the simple solution that does not exploit the sorted order.
3. **Optimized version:** choose the pointers, the loop condition and the movement rule that use the sorted order.
4. **Verification:** dry-run the optimized algorithm and test it on normal, boundary and no-solution inputs.

3.1 Part A: Remove Duplicates (Slow and Fast Pointers)

Reference solution. A separate list `result` is built. Because the input is sorted, a value is a new distinct value exactly when it differs from the last value already stored, so only `result.back()` needs to be checked. The list can grow up to N elements, so the extra space is O(N).

```
function removeDuplicatesReference(nums):
    if nums is empty: return 0
    result = empty list
    for each num in nums:
        if result is empty or last(result) != num:
            append num to result
    return size(result)
```

Optimized solution. The array is reused as its own result. `slow` marks the last position of the confirmed unique prefix, and `fast` scans ahead looking for the next value that differs from `nums[slow]`.

```
function removeDuplicatesOptimized(nums):
    if nums is empty: return 0
    slow = 0
    for fast = 1 to n - 1:
        if nums[fast] != nums[slow]:
            slow = slow + 1
            nums[slow] = nums[fast]
    return slow + 1
```

Table 1: Pointer behaviour of the slow/fast duplicate-removal algorithm

Element	Explanation
Pointer initialization	`slow = 0` (the first element is always unique) and `fast = 1` (the first element to examine).
Loop condition	`fast < n`: fast visits every remaining index exactly once, from left to right.
Comparison condition	`nums[fast] != nums[slow]`: because equal values are adjacent, a value different from the last unique value must be new.
Pointer update	On a new value, `slow` moves one step and `nums[slow]` is overwritten with `nums[fast]`. On a duplicate, only `fast` moves (it advances in every iteration).
Termination	The loop ends when fast reaches n. The unique values are in `nums[0..slow]`, so the answer is `k = slow + 1`. An empty array returns 0 before the loop.

Why it is correct. After every iteration, `nums[0..slow]` holds the distinct values seen so far, in their original order, and `nums[slow]` is the largest of them. Since the array is non-decreasing, a value seen later is either equal to `nums[slow]` or larger, so one comparison is enough. The write to `nums[slow]` never destroys unread data because `slow <= fast` always holds.

3.2 Part B: Pair Sum (Opposite-End Pointers)

Brute-force solution. Every pair (i, j) with i < j is tested. Starting j at i + 1 guarantees that an element is never paired with itself. There are N(N-1)/2 pairs in the worst case.

```
function pairSumBruteForce(nums, target):
    for i = 0 to n - 1:
        for j = i + 1 to n - 1:
            if nums[i] + nums[j] == target: return (i, j)
    return (-1, -1)
```

Optimized solution. `left` starts at the smallest element and `right` at the largest. The sum of the two decides which pointer moves, so one element is discarded in every step.

```
function pairSumOptimized(nums, target):
    left = 0
    right = n - 1
    while left < right:
        currentSum = nums[left] + nums[right]
        if currentSum == target: return (left, right)
        else if currentSum < target: left = left + 1
        else: right = right - 1
    return (-1, -1)
```

Table 2: Pointer behaviour of the opposite-end pair-sum algorithm

Element	Explanation
Pointer initialization	`left = 0` (smallest value) and `right = n - 1` (largest value).
Loop condition	`left < right`: the two pointers must refer to different elements, so an element is never used twice. When they meet, the search window is empty.
Comparison condition	`currentSum = nums[left] + nums[right]` is compared with the target T: equal, smaller or larger.
Pointer update	Sum equal: return (left, right). Sum smaller: `left++`. Sum larger: `right--`.
Termination	Either a pair is found and returned, or `left >= right` and (-1, -1) is returned. At most n - 1 iterations are possible because every iteration moves a pointer.

Why the left pointer moves when the sum is too small

`nums[right]` is the largest value still in the window. If `nums[left] + nums[right] < T`, then pairing `nums[left]` with any other value in the window gives an even smaller sum, because all of them are less than or equal to `nums[right]`. So `nums[left]` can never be part of the answer and is discarded by `left++`.

Why the right pointer moves when the sum is too large

`nums[left]` is the smallest value still in the window. If `nums[left] + nums[right] > T`, then pairing `nums[right]` with any other value in the window gives an even larger sum, because all of them are greater than or equal to `nums[left]`. So `nums[right]` can never be part of the answer and is discarded by `right--`.

Invariant: if a valid pair exists, both of its indices always lie inside [left, right]. Each step removes only an index that cannot belong to any pair, so a pair is never skipped, and the loop ends after at most n - 1 steps.

3.3 Dry-Run Tables

The tables follow the optimized algorithms step by step. The first uses the sample input of Part A, the second the sample input of Part B.

Table 3: Dry run of duplicate removal on [1, 1, 2, 2, 2, 3, 4, 4]

Initial state: slow = 0, array = [1, 1, 2, 2, 2, 3, 4, 4]. The slow and nums[slow] columns show the values at the start of each step; the Action column shows the update that follows.

Step	slow	fast	nums[fast]	nums[slow]	Comparison	Action	Array state after step
1	0	1	1	1	1 == 1	duplicate, skip	[1, 1, 2, 2, 2, 3, 4, 4]
2	0	2	2	1	2 != 1	slow = 1; nums[1] = 2	[1, 2, 2, 2, 2, 3, 4, 4]
3	1	3	2	2	2 == 2	duplicate, skip	[1, 2, 2, 2, 2, 3, 4, 4]
4	1	4	2	2	2 == 2	duplicate, skip	[1, 2, 2, 2, 2, 3, 4, 4]
5	1	5	3	2	3 != 2	slow = 2; nums[2] = 3	[1, 2, 3, 2, 2, 3, 4, 4]
6	2	6	4	3	4 != 3	slow = 3; nums[3] = 4	[1, 2, 3, 4, 2, 3, 4, 4]
7	3	7	4	4	4 == 4	duplicate, skip	[1, 2, 3, 4, 2, 3, 4, 4]

Result: the loop ends with fast = 8 (= n). Return k = slow + 1 = **4**. The unique prefix is **[1, 2, 3, 4]**; the values after index 3 are leftovers and are ignored.

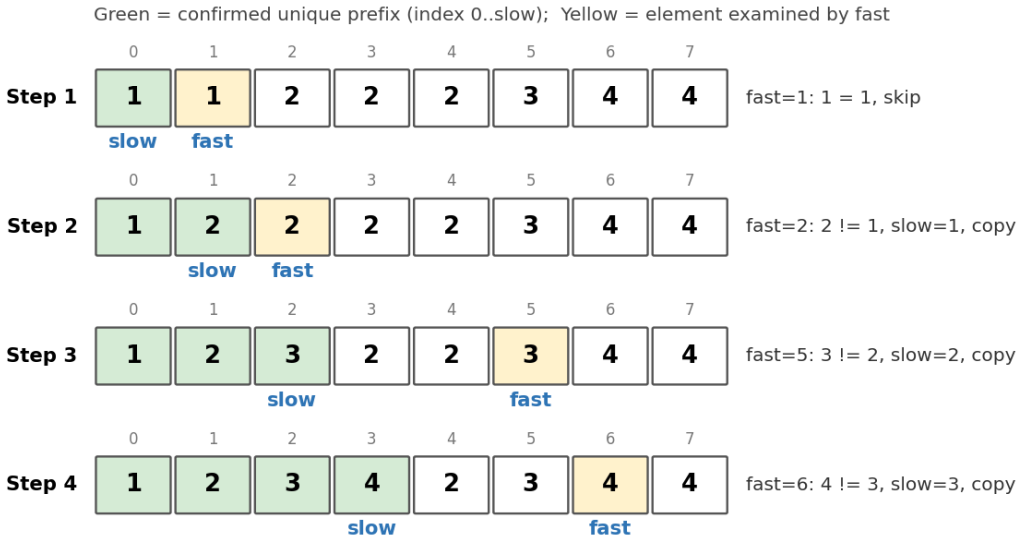


Figure 1: Slow and fast pointer movement on [1, 1, 2, 2, 2, 3, 4, 4]

Key steps only: slow marks the end of the confirmed unique prefix, fast scans ahead.

Table 4: Dry run of pair sum on [1, 2, 4, 7, 11, 15] with target 15

Initial state: left = 0, right = 5.

Step	left	right	nums[left]	nums[right]	Sum	Comparison	Action
1	0	5	1	15	16	16 > 15	sum too large: right--
2	0	4	1	11	12	12 < 15	sum too small: left++
3	1	4	2	11	13	13 < 15	sum too small: left++
4	2	4	4	11	15	15 = 15	pair found, return (2, 4)

Result: the pair at indices **(2, 4)**, with values 4 and 11, is returned after four steps. The array is not modified.

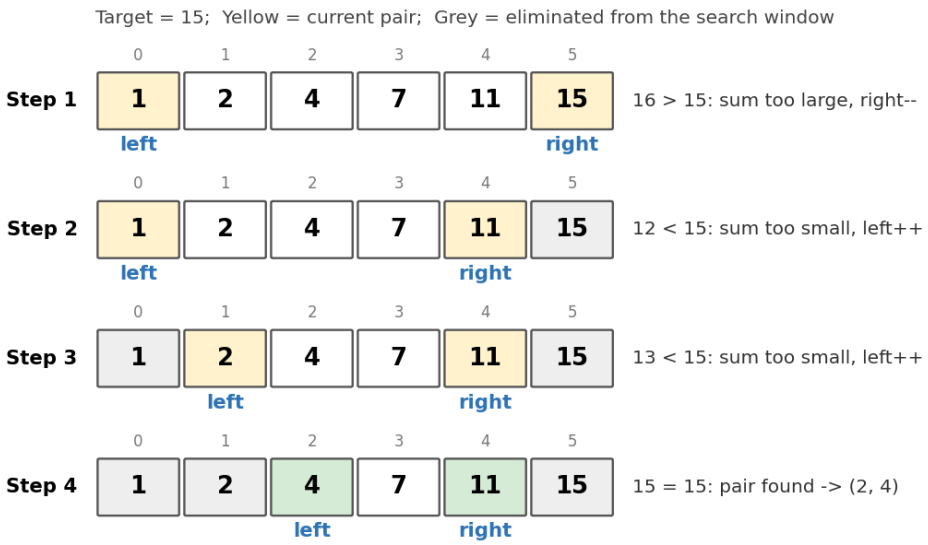


Figure 2: Opposite-end pointer movement on [1, 2, 4, 7, 11, 15], target 15

Left and right close in on each other; grey cells have been eliminated from the search window.

Table 5: Dry run of the no-solution case [2, 4, 6] with target 20

Step	left	right	nums[left]	nums[right]	Sum	Comparison	Action
1	0	2	2	6	8	8 < 20	sum too small: left++
2	1	2	4	6	10	10 < 20	sum too small: left++
3	2	2	-	-	-	left >= right	stop, return (-1, -1)

Even the largest possible sum (4 + 6 = 10) is below 20, so the left pointer keeps moving until it meets the right pointer and (-1, -1) is returned.

3.4 Complexity Comparison

N is the number of elements. Auxiliary space does not count the input array itself.

Table 6: Time and auxiliary-space complexity of all implementations

Implementation	Time	Auxiliary space	Approach	Justification
A: remove duplicates, reference	O(N)	O(N)	Separate list	One pass over N elements with O(1) work each; the result list can hold up to N values (all distinct).
A: remove duplicates, optimized	O(N)	O(1)	Slow/fast pointers	fast visits each index once; only the integers slow and fast are stored, and the array is reused.
B: pair sum, brute force	O(N^2)	O(1)	Nested loops	N(N-1)/2 pairs are tested in the worst case; only the loop counters are stored.
B: pair sum, optimized	O(N)	O(1)	Opposite-end pointers	Every iteration moves one pointer inward, so there are at most N - 1 iterations; only left, right and the sum are stored.

3.5 How Sorted Order Enables the Optimization

Table 7: Reference/brute-force versus optimized approach

Aspect	Reference / brute force	Optimized two-pointer
Part A idea	Copy each new value into a second list.	Overwrite the front of the same array with each new value.
Part A cost	O(N) time, O(N) extra space.	O(N) time, O(1) extra space.
Part B idea	Try every pair of indices.	Compare only the two ends and discard one index per step.
Part B cost	O(N^2) time.	O(N) time.
What sorted order gives	Not used (the brute force would work on unsorted data too).	Part A: equal values are adjacent, so one comparison with the last unique value is enough. Part B: sums grow when left moves right and shrink when right moves left, so each comparison rules out a whole index.
Trade-off	Simpler to write and to trust.	Needs sorted input; on unsorted data the answers are wrong.

4. Methods, Frameworks & Tools Applied in the Assignment

Key methods and concepts:

- **Slow and fast pointers** - two indices moving in the same direction at different speeds; slow marks the write position and fast scans the input.
- **Opposite-end pointers** - two indices that start at the two ends of a sorted array and move towards each other.
- **In-place modification** - the input array is reused as the output so that no extra array is needed.
- **Loop invariants** - statements that stay true in every iteration, used to argue that the algorithms are correct.
- **Brute force and reference solutions** - simple solutions used as a baseline for correctness and for the complexity comparison.
- **Time and auxiliary-space analysis** - Big-O analysis of every implementation with a short justification.
- **Dry runs and test cases** - normal, boundary, duplicate-heavy, single-element, empty and no-solution inputs.

Tools Used for the Assignment

- Languages: C++17 (compiled with g++), Java (javac and java) and Python 3.
- Editor: Visual Studio Code, used for the source files and the code screenshots.
- Data structures: `std::vector` and `std::pair` in C++, `int[]`, `List` and `ArrayList` in Java, lists and tuples in Python.
- Diagrams: Python (matplotlib) for the pointer-movement figures.
- Report format: Word document following the School of Future Tech report template.

5. Implementation Details and Snapshots

The programs were developed in three stages: (1) writing the reference and brute-force versions; (2) writing the optimized pointer versions; (3) testing both versions on the required test cases. The same

four functions exist in all three languages, and each file ends with a short test driver that prints the sample cases.

5.1 Module-wise Implementation

Table 8: Functions in the three source files

Module	C++ (assignment1.cpp)	Java (Assignment1.java)	Python (assignment1.py)
Remove duplicates, reference	`removeDuplicatesReference(const vector<int>&)` returns int	`removeDuplicatesReference(List<Integer>)` returns int	`remove_duplicates_reference(nums)` returns int
Remove duplicates, optimized	`removeDuplicatesOptimized(vector<int>&)` slow/fast, in place	`removeDuplicatesOptimized(int[])` slow/fast, in place	`remove_duplicates_optimized(nums)` slow/fast, in place
Pair sum, brute force	`pairSumBruteForce(nums, target)` returns pair<int,int>	`pairSumBruteForce(int[], int)` returns int[2]	`pair_sum_brute_force(nums, target)` returns tuple
Pair sum, optimized	`pairSumOptimized(nums, target)` returns pair<int,int>	`pairSumOptimized(int[], int)` returns int[2]	`pair_sum_optimized(nums, target)` returns tuple
Test driver	`main()` prints k, the prefix and the pair	`main(String[] args)` prints the same	`if __name__ == "__main__":` prints the same

5.2 Test Cases and Verification

Rows 1-8 are the minimum test cases from the assignment. Rows 9-13 are extra cases added for the single-element, two-element, no-duplicate and all-equal situations. Every case was run with the reference/brute-force and the optimized function of all three languages, and the results were identical. For pair sum, the brute-force and optimized functions returned the same pair in every case.

Table 9: Test-case table: input, expected output, actual output and status

#	Problem	Test input	Expected check	Actual output (C++, Java, Python)	Status
1	Duplicates	[1, 1, 2, 2, 2, 3, 4, 4]	k = 4; prefix [1, 2, 3, 4]	k = 4; prefix [1, 2, 3, 4]	Pass
2	Duplicates	[5]	k = 1; prefix [5]	k = 1; prefix [5]	Pass
3	Duplicates	[]	k = 0; no invalid access	k = 0; prefix []	Pass
4	Duplicates	[-3, -3, -1, 0, 0]	k = 3; prefix [-3, -1, 0]	k = 3; prefix [-3, -1, 0]	Pass
5	Pair sum	[1, 2, 4, 7, 11, 15], T = 15	A valid pair is indices (2, 4)	(2, 4); values (4, 11)	Pass
6	Pair sum	[1, 3, 5, 8], T = 7	Assignment lists (0, 2); see note below	(-1, -1); no pair exists	Pass *
7	Pair sum	[2, 4, 6], T = 20	(-1, -1)	(-1, -1)	Pass
8	Pair sum	[1, 2, 2, 3], T = 4	A valid pair using two different indices	(0, 3); values (1, 3)	Pass
9	Pair sum (extra)	[1, 3, 5, 8], T = 6	(0, 2): 1 + 5 = 6	(0, 2)	Pass
10	Pair sum (extra)	[1, 2], T = 3	(0, 1)	(0, 1)	Pass

#	Problem	Test input	Expected check	Actual output (C++, Java, Python)	Status
11	Pair sum (extra)	[3], T = 3	(-1, -1): one element cannot pair with itself	(-1, -1)	Pass
12	Duplicates (extra)	[1, 2, 3]	k = 3; prefix [1, 2, 3] (no duplicates)	k = 3; prefix [1, 2, 3]	Pass
13	Duplicates (extra)	[7, 7, 7, 7]	k = 1; prefix [7] (all equal)	k = 1; prefix [7]	Pass

Note on test 6 (marked with an asterisk): the assignment lists `[1, 3, 5, 8]`, T = 7 with the expected pair (0, 2), but nums[0] + nums[2] = 1 + 5 = 6, not 7, and no two elements of this array add up to 7 (the possible sums are 4, 6, 8, 9, 11 and 13). The correct answer for T = 7 is therefore (-1, -1), which all three programs return. The expected pair (0, 2) is correct for T = 6, which is checked in test 9. Test 6 is marked as passed on this basis.

Check of the tests: in the duplicate tests, the first k entries of the array are exactly the distinct values in their original order, and the empty array returns k = 0 without reading any element. In the pair-sum tests, every returned pair has i < j and a[i] + a[j] = T, and the no-solution cases return (-1, -1).

5.3 How to Compile and Run

Table 10: Compile and run commands

Language	Compile	Run
C++17	`g++ -std=c++17 -Wall -o assignment1 assignment1.cpp`	`./assignment1`
Java	`javac Assignment1.java` (creates Assignment1.class)	`java Assignment1`
Python 3	Not required	`python3 assignment1.py`

Snapshots

The following exhibits show the pointer movements, the source code of the C++, Java and Python files, and the terminal output of each program.

C++ source code (assignment1.cpp)

assignment1.cpp > main()

```cpp
#include <iostream>
#include <vector>
#include <utility>
#include <algorithm>

// ========================================
// PART A: REMOVE DUPLICATES FROM SORTED ARRAY
// ========================================

// Reference Solution: Uses auxiliary vector (O(N) Space)
int removeDuplicatesReference(const std::vector<int>& nums) {
    if (nums.empty()) return 0;
    std::vector<int> result;
    for (int num : nums) {
        if (result.empty() || result.back() != num) {
            result.push_back(num);
        }
    }
    return result.size();
}

// Optimized Solution: Slow and Fast Pointers (In-Place, O(1) Space)
int removeDuplicatesOptimized(std::vector<int>& nums) {
    if (nums.empty()) return 0;
    int slow = 0;
    for (int fast = 1; fast < (int)nums.size(); fast++) {
        if (nums[fast] != nums[slow]) {
            slow++;
            nums[slow] = nums[fast];
        }
    }
    return slow + 1;
}

// ========================================
// PART B: PAIR SUM IN SORTED ARRAY
// ========================================
```

Figure 3: C++ source code

assignment1.cpp, lines 1-38: Part A (reference and slow/fast pointer solutions)

```cpp

// ========================================
// PART B: PAIR SUM IN SORTED ARRAY
// ========================================

// Brute-Force Solution: Nested loops (O(N^2) Time)
std::pair<int, int> pairSumBruteForce(const std::vector<int>& nums, int target) {
    int n = nums.size();
    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {
            if (nums[i] + nums[j] == target) {
                return {i, j};
            }
        }
    }
    return {-1, -1};
}

// Optimized Solution: Opposite-End Pointers (O(N) Time)
std::pair<int, int> pairSumOptimized(const std::vector<int>& nums, int target) {
    int left = 0;
    int right = (int)nums.size() - 1;
    while (left < right) {
        int currentSum = nums[left] + nums[right];
        if (currentSum == target) {
            return {left, right};
        } else if (currentSum < target) {
            left++;
        } else {
            right--;
        }
    }
    return {-1, -1};
}

int main() {
    std::cout << "=== Part A: Remove Duplicates Test ===" << std::endl;
    std::vector<int> arr1 = {1, 1, 2, 2, 2, 3, 4, 4};
```

Figure 4: C++ source code

assignment1.cpp, lines 34-71: Part B (brute force and opposite-end pointer solutions)

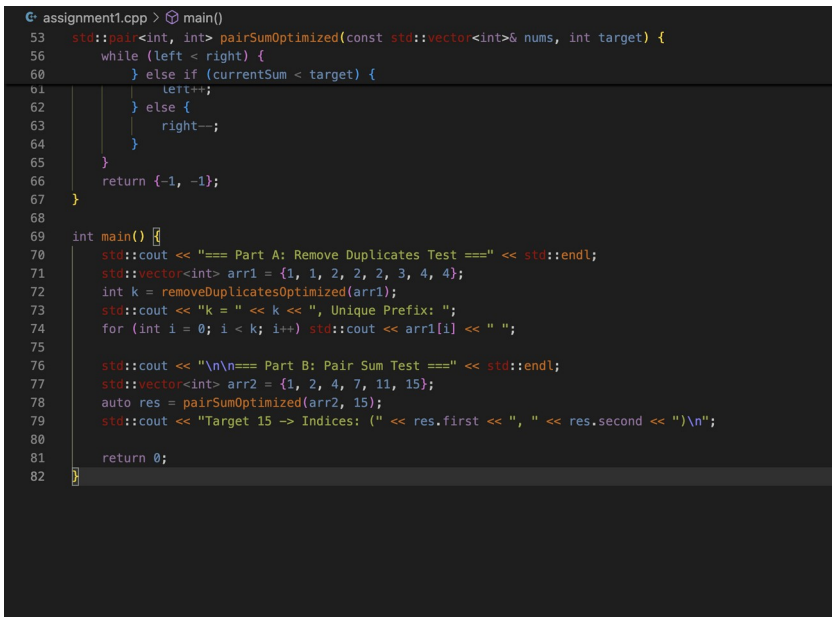


Figure 5: C++ source code

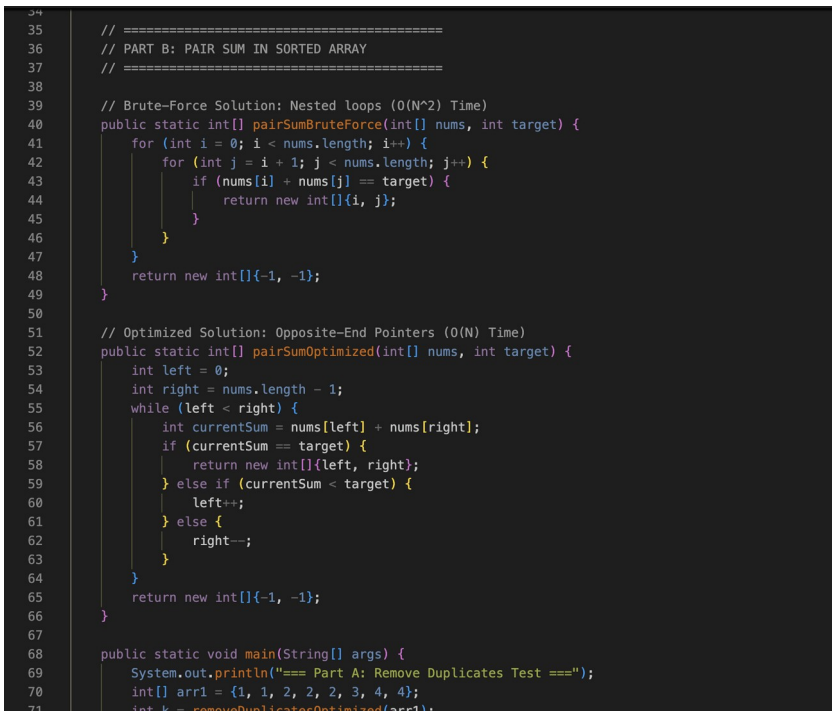
assignment1.cpp, lines 53-82: end of pairSumOptimized and the main() test driver

Java source code (Assignment1.java)



Figure 6: Java source code

Assignment1.java, lines 1-38: Part A (reference and slow/fast pointer solutions)



```java
    // ========================================
    // PART B: PAIR SUM IN SORTED ARRAY
    // ========================================

    // Brute-Force Solution: Nested loops (O(N^2) Time)
    public static int[] pairSumBruteForce(int[] nums, int target) {
        for (int i = 0; i < nums.length; i++) {
            for (int j = i + 1; j < nums.length; j++) {
                if (nums[i] + nums[j] == target) {
                    return new int[]{i, j};
                }
            }
        }
        return new int[]{-1, -1};
    }

    // Optimized Solution: Opposite-End Pointers (O(N) Time)
    public static int[] pairSumOptimized(int[] nums, int target) {
        int left = 0;
        int right = nums.length - 1;
        while (left < right) {
            int currentSum = nums[left] + nums[right];
            if (currentSum == target) {
                return new int[]{left, right};
            } else if (currentSum < target) {
                left++;
            } else {
                right--;
            }
        }
        return new int[]{-1, -1};
    }

    public static void main(String[] args) {
        System.out.println("=== Part A: Remove Duplicates Test ===");
        int[] arr1 = {1, 1, 2, 2, 2, 3, 4, 4};
        int k = removeDuplicatesOptimized(arr1);
```

Figure 7: Java source code

Assignment1.java, lines 35-71: Part B (brute force and opposite-end pointer solutions)



```java
            } else {
                right--;
            }
        }
        return new int[]{-1, -1};
    }

    public static void main(String[] args) {
        System.out.println("=== Part A: Remove Duplicates Test ===");
        int[] arr1 = {1, 1, 2, 2, 2, 3, 4, 4};
        int k = removeDuplicatesOptimized(arr1);
        System.out.print("k = " + k + ", Unique Prefix: ");
        for (int i = 0; i < k; i++) {
            System.out.print(arr1[i] + " ");
        }
        System.out.println("\n\n=== Part B: Pair Sum Test ===");
        int[] arr2 = {1, 2, 4, 7, 11, 15};
        int[] res = pairSumOptimized(arr2, 15);
        System.out.println("Target 15 -> Indices: (" + res[0] + ", " + res[1] + ")");
    }
}
```

Figure 8: Java source code

Assignment1.java, lines 61-81: end of pairSumOptimized and the main method

Python source code (assignment1.py)

assignment1.py > ...

```python
# ========================================
# PART A: REMOVE DUPLICATES FROM SORTED ARRAY
# ========================================

# Reference Solution: Uses auxiliary list (O(N) Space)
def remove_duplicates_reference(nums):
    if not nums:
        return 0
    result = []
    for num in nums:
        if not result or result[-1] != num:
            result.append(num)
    return len(result)

# Optimized Solution: Slow and Fast Pointers (In-Place, O(1) Space)
def remove_duplicates_optimized(nums):
    if not nums:
        return 0
    slow = 0
    for fast in range(1, len(nums)):
        if nums[fast] != nums[slow]:
            slow += 1
            nums[slow] = nums[fast]
    return slow + 1


# ========================================
# PART B: PAIR SUM IN SORTED ARRAY
# ========================================

# Brute-Force Solution: Nested loops (O(N^2) Time)
def pair_sum_brute_force(nums, target):
    n = len(nums)
    for i in range(n):
        for j in range(i + 1, n):
            if nums[i] + nums[j] == target:
                return (i, j)
    return (-1, -1)
```

Figure 9: Python source code

assignment1.py, lines 1-38: Part A and the brute-force pair sum

```python
# ========================================
# PART B: PAIR SUM IN SORTED ARRAY
# ========================================

# Brute-Force Solution: Nested loops (O(N^2) Time)
def pair_sum_brute_force(nums, target):
    n = len(nums)
    for i in range(n):
        for j in range(i + 1, n):
            if nums[i] + nums[j] == target:
                return (i, j)
    return (-1, -1)

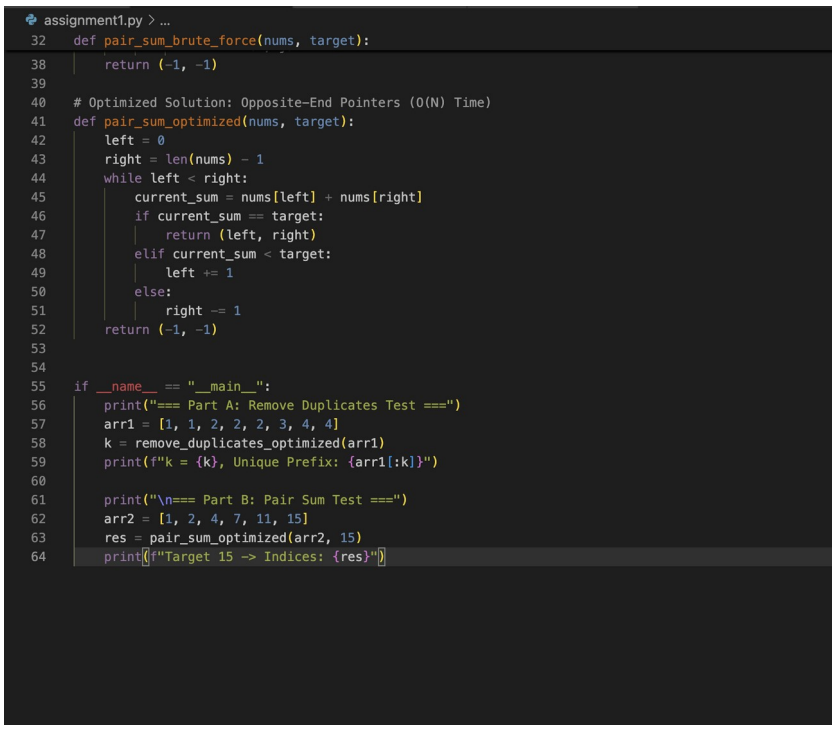
# Optimized Solution: Opposite-End Pointers (O(N) Time)
def pair_sum_optimized(nums, target):
    left = 0
    right = len(nums) - 1
    while left < right:
        current_sum = nums[left] + nums[right]
        if current_sum == target:
            return (left, right)
        elif current_sum < target:
            left += 1
        else:
            right -= 1
    return (-1, -1)


if __name__ == "__main__":
    print("=== Part A: Remove Duplicates Test ===")
    arr1 = [1, 1, 2, 2, 2, 3, 4, 4]
    k = remove_duplicates_optimized(arr1)
    print(f"k = {k}, Unique Prefix: {arr1[:k]}")

    print("\n=== Part B: Pair Sum Test ===")
    arr2 = [1, 2, 4, 7, 11, 15]
    res = pair_sum_optimized(arr2, 15)
    print(f"Target 15 -> Indices: {res}")
```

Figure 10: Python source code

assignment1.py, lines 27-64: Part B and the __main__ test driver



```python
# assignment1.py > ...
def pair_sum_brute_force(nums, target):
    return (-1, -1)

# Optimized Solution: Opposite-End Pointers (O(N) Time)
def pair_sum_optimized(nums, target):
    left = 0
    right = len(nums) - 1
    while left < right:
        current_sum = nums[left] + nums[right]
        if current_sum == target:
            return (left, right)
        elif current_sum < target:
            left += 1
        else:
            right -= 1
    return (-1, -1)


if __name__ == "__main__":
    print("=== Part A: Remove Duplicates Test ===")
    arr1 = [1, 1, 2, 2, 2, 3, 4, 4]
    k = remove_duplicates_optimized(arr1)
    print(f"k = {k}, Unique Prefix: {arr1[:k]}")

    print("\n=== Part B: Pair Sum Test ===")
    arr2 = [1, 2, 4, 7, 11, 15]
    res = pair_sum_optimized(arr2, 15)
    print(f"Target 15 -> Indices: {res}")
```

Figure 11: Python source code

assignment1.py, lines 32-64: pair_sum_optimized and the __main__ test driver

Terminal output

Copied terminal output of the three programs, showing the compile command (where needed), the run command and the printed results.

```
$ g++ -std=c++17 -Wall -o assignment1 assignment1.cpp
$ ./assignment1
=== Part A: Remove Duplicates Test ===
k = 4, Unique Prefix: 1 2 3 4

=== Part B: Pair Sum Test ===
Target 15 -> Indices: (2, 4)
```

Figure 12: C++ compilation and execution

```
$ javac Assignment1.java
$ java Assignment1
=== Part A: Remove Duplicates Test ===
k = 4, Unique Prefix: 1 2 3 4

=== Part B: Pair Sum Test ===
Target 15 -> Indices: (2, 4)
```

Figure 13: Java compilation and execution

```
$ python3 assignment1.py
=== Part A: Remove Duplicates Test ===
k = 4, Unique Prefix: [1, 2, 3, 4]

=== Part B: Pair Sum Test ===
Target 15 -> Indices: (2, 4)
```

Figure 14: Python execution

6. Assignment Results and Conclusion

Findings

- All four algorithms (two for each problem) were implemented in C++, Java and Python, and each language gives the same result for every test case.
- The in-place slow/fast pointer solution returns k = 4 with the prefix [1, 2, 3, 4] for the sample input, and handles the empty array (k = 0), the one-element array (k = 1) and negative values.
- The opposite-end pointer solution finds the pair (2, 4) for the sample input after four steps, returns (-1, -1) when no pair exists, and never pairs an element with itself.
- The optimized duplicate removal needs O(1) extra space against O(N) for the reference version; the optimized pair sum needs O(N) time against O(N^2) for the brute force.
- The brute-force and optimized pair-sum functions agreed on every test case, which supports the correctness argument based on the loop invariant.
- One expected value in the assignment's test table (test 6) is inconsistent with its own input; the programs return the mathematically correct result.

Submission Checklist

Table 11: Check of the submission checklist

Checklist item	Status	Evidence
Optimized duplicate removal modifies the array in place and returns the unique count	Met	Overwrites nums[slow]; returns slow + 1 (Section 3.1, Table 3).
Pair sum never pairs an element with itself and returns (-1, -1) when no pair exists	Met	Loop condition left < right; brute force starts j at i + 1 (Tests 3, 7 and 11).
All required test cases pass	Met	Tests 1-8 in Table 9; test 6 explained in the note.
Report matches the submitted code	Met	Function names and logic in Sections 3 and 5 match the screenshots.
Student name and roll number included	Met	Name, roll number and semester are on the cover page.

Conclusion

This assignment shows that the sorted order of an array can be turned directly into a faster or leaner algorithm. For duplicate removal, equal values sit next to each other, so a slow pointer that marks the end of the unique prefix and a fast pointer that scans ahead are enough to compact the array in one pass with no extra memory. For pair sum, the sum of the two end values tells which end can be discarded, so the quadratic search collapses into a single linear pass. In both cases the optimized version gave exactly the same answers as the simple version on all test cases, so the assignment meets its objectives.

Scope for Improvement

- Use `long long` (C++) or `long` (Java) for the sum in the pair-sum functions so that very large values cannot overflow an `int`.
- Read the arrays and the target from the user (standard input) instead of using fixed sample data in the test driver.
- Add an automatic test harness that checks every test case and prints pass or fail, instead of running them one by one.

- Extend pair sum to return all pairs, or all distinct value pairs, with the same pointer technique.
- Sort the input first (O(N log N)) or use a hash-based method so that the same functions also work on unsorted arrays.

7. References

- DSA-II Programming Assignment 1: Two-Pointer Technique, School of Future Tech, ITM Skills University, SFT AY 2026-27, B.Tech CSE.
- Course material on Data Structures and Algorithms II, Semester III: arrays and the two-pointer technique.
- C++ Reference: `std::vector`, `std::pair` (cppreference.com).
- Java Platform SE documentation: `java.util.List` and `java.util.ArrayList`.
- Python documentation: lists, tuples and f-strings (docs.python.org).